

MATERIAL DE ESTUDO · THREE.JS

Portfólio 3D com Robô

Guia de Estudo Completo

Os 9 tópicos técnicos necessários para construir seu portfólio interativo — cada um com o conceito, um diagrama explicativo, um exemplo ilustrativo de código e a aplicação direta no seu projeto: um robô que anda por um mapa 3D revelando suas experiências em robótica.

Lucas Aniceto Bárbara de Souza

Amigos Droids · FTC #17792

Colégio Cotemig

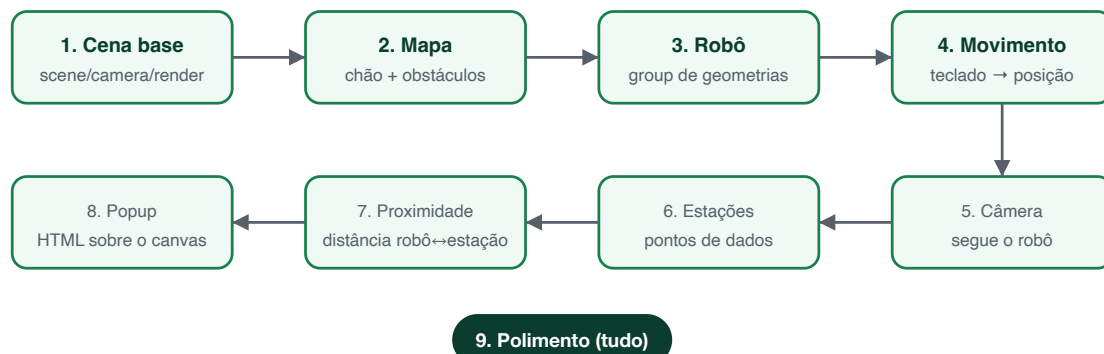
Como usar este documento

Cada tópico corresponde a uma etapa do roteiro de estudo sugerido. Siga a ordem — cada etapa depende de conceitos da anterior. Em cada tópico você vai encontrar quatro blocos fixos:

Bloco	O que contém
Conceito	O que é, por que existe, como o Three.js resolve o problema
Diagrama	Visualização do conceito (esquema, não é print de código)
Exemplo ilustrativo	Trecho curto de código para fixar a sintaxe — não é o código final do seu projeto
Aplicação no seu projeto	Como esse conceito específico entra no robô/mapa/popups que você vai construir

Os 9 tópicos

- | | | |
|---|--|--------|
| 1 | Fundamentos do Three.js (cena, câmera, renderer) | ≈ 3–4h |
| 2 | Mapa e chão básico | ≈ 1–2h |
| 3 | Robô com geometrias simples | ≈ 2–3h |
| 4 | Movimento por teclado | ≈ 2–3h |
| 5 | Câmera seguindo o robô | ≈ 1–2h |
| 6 | Estações fixas no mapa | ≈ 1–2h |
| 7 | Deteção de proximidade | ≈ 1–2h |
| 8 | Popup HTML conectado | ≈ 2–3h |
| 9 | Polimento visual | ≈ 4–6h |



Fluxo de dependências entre os 9 tópicos — cada bloco usa o que foi construído no anterior.

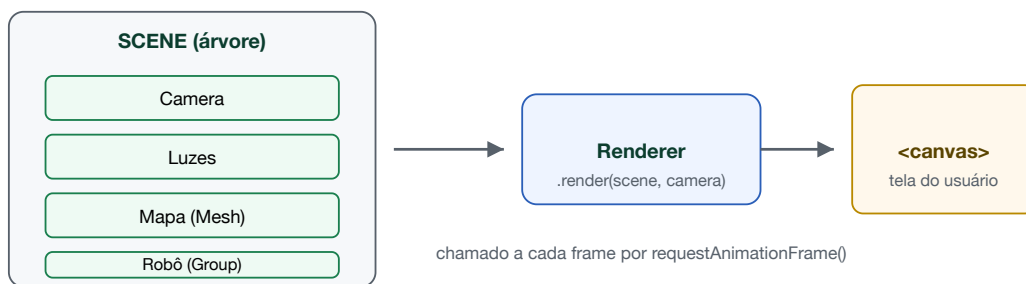
Fundamentos do Three.js

Scene, Camera, Renderer e o loop de renderização — o "motor" que faz tudo existir na tela.

Conceito

Three.js não desenha nada sozinho: ele é uma camada sobre o WebGL que organiza três peças obrigatórias.

- **Scene (`THREE.Scene`)** — o "container" que guarda tudo: robô, mapa, luzes, estações. É uma árvore (grafo de cena): cada objeto pode ter objetos "filhos" dentro dele.
- **Camera** — define o ponto de vista. As duas mais usadas: `PerspectiveCamera` (com profundidade, como o olho humano) e `OrthographicCamera` (sem distorção de distância — usada em jogos isométricos).
- **Renderer (`WebGLRenderer`)** — pega a Scene + a Camera e "tira uma foto", desenhando no `<canvas>`.
- **Loop de renderização** — a "foto" precisa ser tirada de novo a cada frame (idealmente 60x por segundo) para parecer animado. Isso é feito com `requestAnimationFrame`, uma função nativa do navegador que chama sua função de novo antes do próximo repaint da tela.



A cena é uma árvore de objetos; o renderer "fotografa" essa árvore do ponto de vista da câmera, a cada frame.

Exemplo ilustrativo

```
// estrutura mínima — não é o código final do projeto, é para fixar a sintaxe
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(50, innerWidth/innerHeight, 0.1, 1000);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

camera.position.set(0, 5, 10);

function animate() {
  requestAnimationFrame(animate);
  renderer.render(scene, camera);
}
animate();
```

APLICAÇÃO NO SEU PROJETO

Esse bloco vira o seu `main.js`: o arquivo que cria a `scene`, a `camera` e o `renderer` uma única vez, e mantém o `animate()` rodando para sempre — é dentro desse loop que, mais adiante, você vai chamar

"atualizar posição do robô", "checar distância das estações" e "atualizar câmera" a cada frame. Todos os outros 8 tópicos deste documento são coisas que entram *dentro* dessa estrutura, não substituem ela.

Aprofunde

threejs.org/docs → busque "Scene", "PerspectiveCamera", "WebGLRenderer". Exemplo oficial:
threejs.org/examples/#webgl_animation_cloth (veja o padrão do loop, ignore o resto).

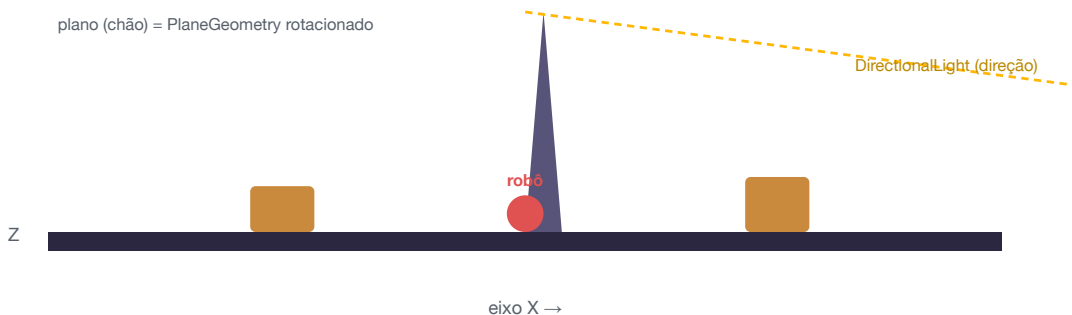
Mapa e chão básico

Geometria plana, materiais, luz e obstáculos — o "terreno" onde o robô anda.

Conceito

Todo objeto visível no Three.js é um **Mesh** = **Geometry** (a forma) + **Material** (a aparência).

- **PlaneGeometry** — um retângulo plano. Por padrão fica "de pé" (encarando a câmera); para virar chão, você rotaciona -90° no eixo X.
- **Materiais** — `MeshBasicMaterial` ignora luz (cor sempre igual); `MeshStandardMaterial` reage à luz (sombra, brilho) — é o que dá aparência realista.
- **Luzes** — sem luz, materiais "Standard" ficam pretos. O par básico é `AmbientLight` (ilumina tudo igualmente, sem sombra) + `DirectionalLight` (luz "do sol", cria sombras e volume).
- **Obstáculos** — caixas (`BoxGeometry`) simples espalhadas pelo mapa, cada uma é só mais um Mesh adicionado à cena com uma posição.



Vista de cima do mapa: chão plano, obstáculos (caixas) e robô, com luz direcional vindo de um ângulo.

Exemplo ilustrativo

```
// chão
const floor = new THREE.Mesh(
  new THREE.PlaneGeometry(50, 50),
  new THREE.MeshStandardMaterial({ color: 0x1b1440 })
);
floor.rotation.x = -Math.PI / 2;
scene.add(floor);

// luzes
scene.add(new THREE.AmbientLight(0xffffff, 0.6));
const sun = new THREE.DirectionalLight(0xffffff, 1);
sun.position.set(10, 15, 10);
scene.add(sun);

// um obstáculo
const box = new THREE.Mesh(
  new THREE.BoxGeometry(2, 2, 2),
  new THREE.MeshStandardMaterial({ color: 0xc98a3e })
);
```

```
box.position.set(5, 1, -3);  
scene.add(box);
```

APLICAÇÃO NO SEU PROJETO

Este é o seu `map.js`. Como você quer uma estética "arena/tech" (referenciando o print que você me mostrou, com aquele rio neon), o chão pode ser um plano escuro com um segundo plano fino e emissivo (brilhante) representando uma "linha de energia" ou "grid" — a mesma técnica de material, só com `emissive` ligado. As caixas viram os obstáculos decorativos ao redor das estações. Pense no tamanho do mapa em função de quantas estações (tópico 6) você vai ter — não precisa ser gigante, 30x30 unidades já é suficiente para 5–6 pontos de interesse.

Aprofunde

threejs.org/docs → "PlaneGeometry", "MeshStandardMaterial", "DirectionalLight". Exemplo oficial:
threejs.org/examples/#webgl_lights_hemisphere.

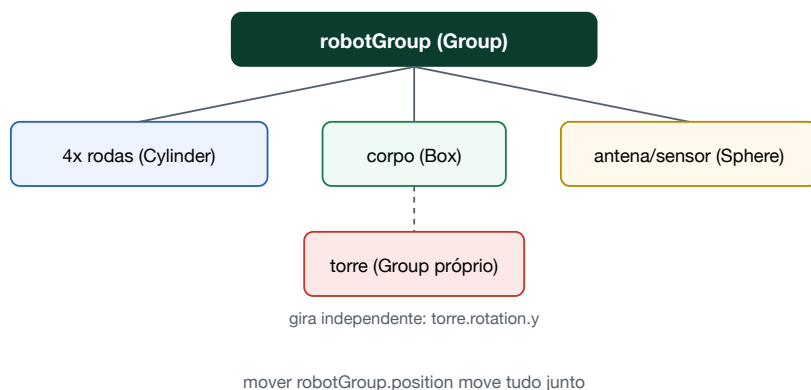
Robô com geometrias simples

Group, hierarquia de objetos e pivôs — como montar um "personagem" a partir de formas básicas.

Conceito

Um robô não é um Mesh único — é vários Meshes (corpo, rodas, antena) organizados dentro de um `THREE.Group`, que você move e rotaciona como uma unidade só.

- **Group** — um "container" que não é visível sozinho, mas agrupa outros objetos. Mover/rotacionar o Group move/rotaciona todos os filhos juntos, mantendo a posição relativa entre eles.
- **Posição local vs. global** — cada Mesh dentro do Group tem uma posição *relativa ao Group*, não ao mundo. Isso é o que permite, por exemplo, colocar 4 rodas nos "cantos" do corpo sem recalculá-las toda vez que o robô se mover.
- **Pivô** — o ponto em torno do qual um objeto gira. Importante se você quiser, por exemplo, uma peça do robô (como uma torre) girando de forma independente do corpo — ela precisa ser um Group/Mesh próprio, filho do robô, com sua própria rotação.



Hierarquia do robô: um Group "pai" contendo as partes; uma peça (torre) pode ter rotação própria, independente do corpo.

Exemplo ilustrativo

```
const robot = new THREE.Group();

const body = new THREE.Mesh(
  new THREE.BoxGeometry(1.2, 0.6, 1.6),
  new THREE.MeshStandardMaterial({ color: 0x8a1c3a })
);
body.position.y = 0.5;
robot.add(body);

const wheelGeo = new THREE.CylinderGeometry(0.3, 0.3, 0.2, 16);
[[-0.6,0.3,0.8], [0.6,0.3,0.8], [-0.6,0.3,-0.8], [0.6,0.3,-0.8]]
  .forEach(([x,y,z]) => {
    const wheel = new THREE.Mesh(wheelGeo, new THREE.MeshStandardMaterial({ color: 0x111111 }));
    wheel.rotation.z = Math.PI / 2;
    wheel.position.set(x,y,z);
    robot.add(wheel);
  });
```

```
scene.add(robot); // mover robot.position move o conjunto inteiro
```

APLICAÇÃO NO SEU PROJETO

Aqui é onde entra a sua identidade visual de robótica. Ideia concreta: monte o corpo como um chassi retangular baixo, 4 rodas/esteiras nas laterais, e adicione uma "torre" (cilindro fino ou caixa) no topo que gira sozinha — uma referência direta ao seu **sistema de torre rotativa com controle PID** do currículo. Adicione também um pequeno retângulo/lente na frente representando a **câmera Limelight** de visão computacional, com um material emissivo (brilhante) para destacar. Esse é o tipo de detalhe que conecta o visual do projeto com sua experiência real, sem precisar de um modelo 3D complexo.

Aprofunde

threejs.org/docs → "Group", "Object3D" (toda a hierarquia de posição/rotação vem daqui). Exemplo oficial: threejs.org/examples/#webgl_geometry_shapes.

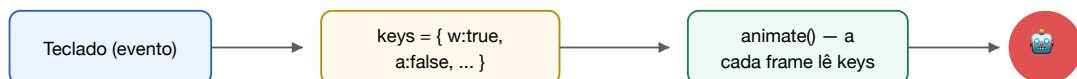
Movimento por teclado

Capturar input, guardar estado das teclas e mover o robô de forma independente da taxa de quadros.

Conceito

Você *nunca* move o robô diretamente dentro do evento de teclado. Você guarda "quais teclas estão pressionadas agora" e aplica o movimento a cada frame, dentro do loop de animação.

- **Eventos** `keydown` / `keyup` — disparados pelo navegador quando uma tecla é pressionada/solta. Você usa isso só para atualizar um objeto de estado (ex: `{ w: true, a: false, ... }`), nunca para mover o robô ali mesmo.
- **Por que não mover direto no evento?** Eventos de teclado não disparam numa taxa constante (dependem do sistema operacional). Se você mover o robô ali, o movimento fica "engasgado". Mover dentro do loop de 60fps garante suavidade.
- **Delta time** — o tempo real entre um frame e o outro (via `THREE.Clock`). Multiplicar a velocidade pelo delta garante que o robô ande na mesma velocidade em qualquer computador, independente do FPS.



o evento só atualiza o "estado"; quem move o robô é sempre o loop de 60fps

Fluxo: evento de teclado atualiza um objeto de estado; o loop de renderização lê esse estado a cada frame e move o robô.

Exemplo ilustrativo

```
const keys = {};  
addEventListener('keydown', e => keys[e.key.toLowerCase()] = true);  
addEventListener('keyup', e => keys[e.key.toLowerCase()] = false);  
  
const clock = new THREE.Clock();  
const speed = 4; // unidades por segundo  
  
function animate() {  
  requestAnimationFrame(animate);  
  const delta = clock.getDelta(); // segundos desde o frame anterior  
  
  if (keys['w']) robot.position.z -= speed * delta;  
  if (keys['s']) robot.position.z += speed * delta;  
  if (keys['a']) robot.rotation.y += 2 * delta;  
  if (keys['d']) robot.rotation.y -= 2 * delta;  
  
  renderer.render(scene, camera);  
}
```

APLICAÇÃO NO SEU PROJETO

Esse é o coração da "jogabilidade" do portfólio. Sugestão de esquema de controle: W/S andam para frente/trás na direção que o robô está olhando (não apenas nos eixos X/Z fixos), A/D rotacionam — é o clássico "controle de carro/tanque", que combina com a estética do exemplo que você mandou. Vale girar também as rodas do robô (rotacioná-las no próprio eixo, proporcional à velocidade) — é um detalhe pequeno que aumenta muito a sensação de "isso é um robô de verdade andando", reforçando a conexão com seus sistemas de controle de motores.

Aprofunde

MDN → "KeyboardEvent", "requestAnimationFrame". threejs.org/docs → "Clock".

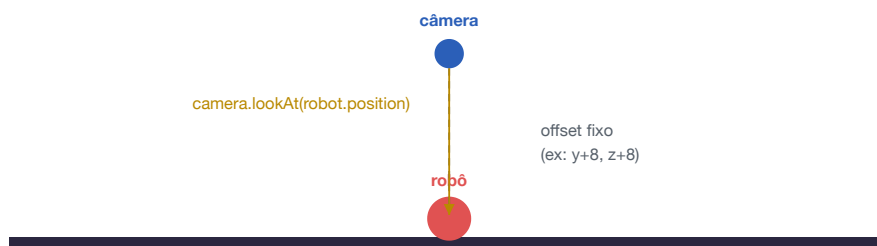
Câmera seguindo o robô

Perspectiva vs. ortográfica, offset fixo e interpolação suave (lerp).

Conceito

A câmera não é "presa" ao robô por hierarquia — a cada frame, você recalcula a posição dela como **posição do robô + um deslocamento (offset) fixo**, e aponta ela para o robô.

- **PerspectiveCamera** — objetos distantes parecem menores (como os olhos). Boa para senso de profundidade.
- **OrthographicCamera** — não há distorção por distância; objetos do mesmo tamanho aparecem do mesmo tamanho na tela não importa a distância. É essa que dá aquele visual "isométrico/jogo de estratégia" do print que você mandou.
- **lookAt()** — método que rotaciona a câmera para "olhar" para um ponto/objeto específico.
- **lerp (interpolação linear)** — em vez de a câmera "teleportar" instantaneamente para a posição ideal a cada frame, ela se move uma fração da distância por frame, criando um movimento suave e "elástico" atrás do robô.



A cada frame: câmera se move em direção a "posição do robô + offset", e sempre olha para o robô.

Exemplo ilustrativo

```
// dentro de animate(), depois de mover o robô:  
const offset = new THREE.Vector3(0, 8, 8);  
const targetPos = robot.position.clone().add(offset);  
  
camera.position.lerp(targetPos, 0.08); // 0.08 = "suavidade" do seguimento  
camera.lookAt(robot.position);
```

APLICAÇÃO NO SEU PROJETO

Para reproduzir o visual do print que você enviou, use `OrthographicCamera` com um ângulo fixo tipo "3/4 de cima" (offset com Y bem maior que Z, por exemplo (0, 10, 6)). Comece com valores de lerp baixos (0.05–0.1) — parece sutil no código mas faz diferença enorme na sensação de "câmera viva" ao apresentar o projeto para a banca. Se notar que a câmera "atravessa" obstáculos ou perde o robô de vista em cantos do mapa, é sinal de que vale limitar os limites do mapa (não deixar o robô sair da área visível).

Aprofunde

threejs.org/docs → "OrthographicCamera", "Vector3.lerp". Termo para buscar: "third person camera follow three.js".

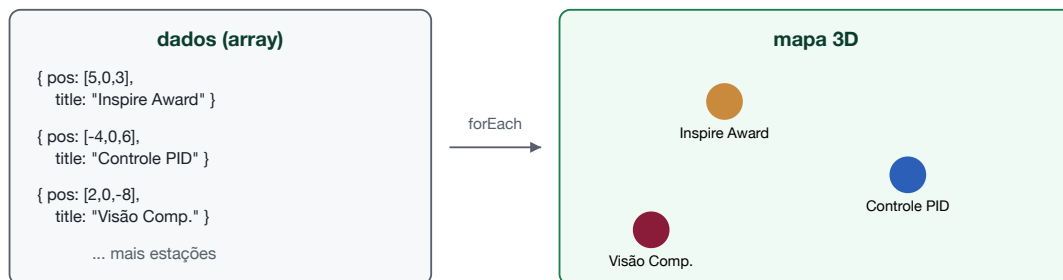
Estações fixas no mapa

Design orientado a dados: um array com o conteúdo do seu portfólio, renderizado como objetos no mapa.

Conceito

Em vez de criar cada marcador "na mão" e repetir código, você separa **dados** (o conteúdo: título, texto, posição) de **lógica** (o código que lê esses dados e cria os objetos 3D). Isso é o padrão mais usado em qualquer projeto real.

- Um **array de objetos JavaScript**, cada um representando uma estação: posição no mapa, título, descrição, talvez uma cor/ícone.
- Um **loop** (`forEach`) que percorre esse array e, para cada item, cria um Mesh (o "marcador" visual) na posição indicada — e guarda uma referência ao dado original junto com o objeto 3D, para usar depois na detecção de proximidade e no popup.
- Vantagem prática: para adicionar uma nova estação/projeto ao portfólio, você só adiciona uma linha nos dados — não escreve código 3D novo.



Os dados do currículo viram um array; o código percorre o array e cria um marcador 3D para cada item.

Exemplo ilustrativo

```
const stations = [
  { position: [6,0,4], title: "Campeão Mundial FTC 2026", text: "Inspire Award, Houston..." },
  { position: [-5,0,7], title: "Controle PID", text: "Torre rotativa de precisão..." },
  { position: [3,0,-9], title: "Visão Computacional", text: "Câmera Limelight + rede neural..." },
];

stations.forEach(data => {
  const marker = new THREE.Mesh(
    new THREE.CylinderGeometry(0.5,0.5,1.5,8),
    new THREE.MeshStandardMaterial({ color: 0xfd8b0, emissive: 0x1a7f4e })
  );
  marker.position.set(...data.position);
  marker.userData = data; // guarda o conteúdo junto com o objeto 3D
  scene.add(marker);
});
```

APLICAÇÃO NO SEU PROJETO

Este array é literalmente o **seu currículo estruturado em dados**. Cada item do seu resumo profissional vira uma estação: "Campeão Mundial FTC 2026", "Controle PID", "Visão Computacional com Limelight", "Simulação da Arena", "Stack técnica (Java/Python/C#)", "Formação Cotemig". Pense na distribuição espacial como uma "linha do tempo" ou "trilha" pelo mapa — o robô percorrendo fisicamente sua trajetória de aprendizado é uma metáfora forte para a apresentação escolar.

Aprofunde

MDN → "Array.prototype.forEach". threejs.org/docs → "Object3D.userData" (campo livre para guardar dados customizados em qualquer objeto 3D).

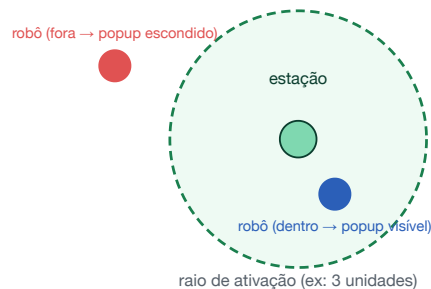
Detecção de proximidade

Matemática simples de distância entre pontos — sem física, sem colisão "de verdade".

Conceito

Você não precisa de um motor de física para saber se o robô está "perto o suficiente" de uma estação — basta calcular a **distância entre dois pontos no espaço 3D** e comparar com um raio de ativação.

- `Vector3.distanceTo()` — método pronto do Three.js que calcula a distância euclidiana entre duas posições.
- **Raio de ativação** — um número (ex: 3 unidades) que define "perto o suficiente". Se `distância < raio`, o robô está dentro da zona da estação.
- Isso é feito **a cada frame**, dentro do loop de animação, para cada estação — com poucas estações (5–8), isso é extremamente barato computacionalmente, não se preocupe com performance aqui.
- **Alternativa mais avançada (raycasting)** — dispara um "raio" invisível e detecta o que ele intercepta; é mais usado para cliques do mouse em objetos 3D. Para "o robô está perto de X", distância simples é a ferramenta certa — mais simples e mais previsível.



Se a distância robô↔estação for menor que o raio, a estação é considerada "ativa".

Exemplo ilustrativo

```
// dentro de animate(), a cada frame:
const ACTIVATION_RADIUS = 3;
let activeStation = null;

stationMarkers.forEach(marker => {
  const dist = robot.position.distanceTo(marker.position);
  if (dist < ACTIVATION_RADIUS) {
    activeStation = marker.userData; // os dados guardados no tópico 6
  }
});

if (activeStation) {
  showPopup(activeStation); // tópico 8
} else {
  hidePopup();
}
```

APLICAÇÃO NO SEU PROJETO

O raio de ativação é um número que vale a pena ajustar testando ao vivo: pequeno demais e fica difícil "acertar" a estação; grande demais e mais de uma estação fica ativa ao mesmo tempo (o que gera ambiguidade sobre qual popup mostrar). Um bom ponto de partida é raio $\approx 1,5x$ o tamanho do robô. Se quiser um efeito visual extra sem esforço, aproveite esse mesmo cálculo de distância para fazer o marcador da estação "pulsar" ou brilhar mais forte conforme o robô se aproxima — reaproveita o número que você já está calculando.

Aprofunde

threejs.org/docs → "Vector3" (métodos `distanceTo`, `distanceToSquared` — esse último é ainda mais rápido, evita a raiz quadrada, útil se algum dia tiver dezenas de estações).

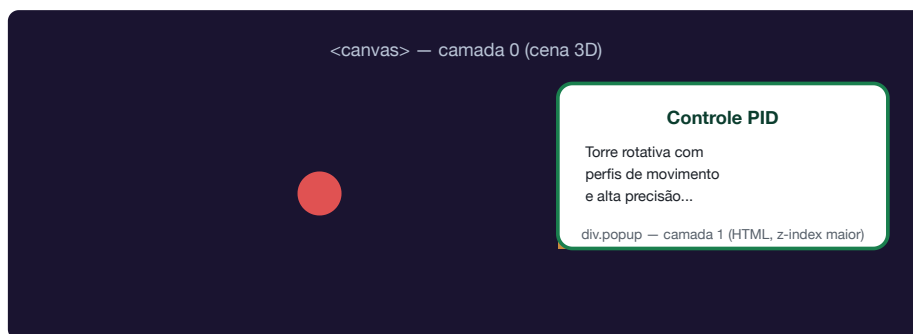
Popup HTML conectado

Uma camada HTML/CSS comum, sobreposta ao canvas — não faz parte da cena 3D.

Conceito

O popup **não é um objeto 3D**. É um elemento HTML normal (`<div>`), posicionado sobre o `<canvas>` com CSS, cujo conteúdo e visibilidade são controlados pelo JavaScript de acordo com o resultado do tópico 7.

- **Camadas (z-index)** — o canvas do Three.js e a `<div>` do popup ocupam o mesmo espaço da tela, mas em camadas diferentes; CSS (`position: absolute` + `z-index`) garante que o popup fique visualmente por cima.
- **Atualizar conteúdo** — quando uma estação fica ativa, você escreve o título/texto dela dentro da div (via `textContent` / `innerHTML`) e alterna a visibilidade (`classList.add/remove` ou `style.display`).
- **Extra opcional — popup "grudado" no objeto 3D**: dá para converter a posição 3D da estação em coordenadas de tela (`Vector3.project(camera)`) e posicionar a div exatamente ali, criando um popup que acompanha o objeto na tela. Não é necessário para o projeto funcionar — um popup fixo (ex: canto da tela) já resolve bem e é mais simples de estilizar.



O popup é HTML comum sobreposto ao canvas — controlado pelo mesmo JavaScript que já sabe "qual estação está ativa".

Exemplo ilustrativo

```
<!-- index.html -->
<div id="popup" class="hidden">
  <h3 id="popup-title"></h3>
  <p id="popup-text"></p>
</div>

/* style.css */
#popup { position: absolute; top: 20px; right: 20px; z-index: 10; }
.hidden { display: none; }

// JS — chamado a partir do tópico 7
function showPopup(data) {
  document.getElementById('popup-title').textContent = data.title;
  document.getElementById('popup-text').textContent = data.text;
  document.getElementById('popup').classList.remove('hidden');
}
function hidePopup() {
```

```
document.getElementById('popup').classList.add('hidden');  
}
```

APLICAÇÃO NO SEU PROJETO

Esse popup é a "vitrine" de cada experiência do currículo — pode incluir título, descrição, e até um link (ex: repositório do código da equipe, se público, ou uma foto da competição em Houston). Como o conteúdo já está estruturado como dados (tópico 6), o mesmo componente de popup serve para todas as estações — você só troca o conteúdo. Para a apresentação na escola, vale caprichar no popup da conquista principal (Campeão Mundial / Inspire Award), já que é o destaque do currículo.

Aprofunde

MDN → "position: absolute", "z-index", "classList". Opcional avançado: threejs.org/docs → "Vector3.project" (para popups "grudados" no objeto 3D).

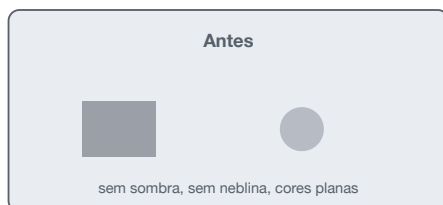
Polimento visual

Sombras, neblina, materiais emissivos e responsividade — o que separa um protótipo de uma apresentação de impacto.

Conceito

Esta etapa não adiciona funcionalidade nova — refina o que já existe. É onde o projeto passa de "funciona" para "impressiona", e normalmente é a etapa que mais tempo consome.

- **Sombras** — precisam ser ligadas explicitamente: `renderer.shadowMap.enabled = true`, e cada Mesh precisa declarar se `castShadow` (projeta sombra) e/ou `receiveShadow` (recebe sombra). Sem isso, tudo parece "flutuando".
- **Neblina (`THREE.Fog`)** — esconde gradualmente objetos distantes na cor de fundo, dando profundidade e escondendo as "bordas" do mapa — muito usado em cenas noturnas/estilizadas como a do seu exemplo.
- **Materiais emissivos** — a propriedade `emissive` faz uma superfície parecer "brilhar com luz própria" mesmo sem receber luz — é a técnica por trás dos detalhes neon do print (faróis, linhas do rio, marcadores de estação).
- **Responsividade** — escutar o evento `resize` da janela e atualizar o aspect ratio da câmera + tamanho do renderer, senão a cena distorce ao redimensionar (importante se for apresentar em telas/projetores diferentes).
- **Pós-processamento (opcional, avançado)** — efeitos como *bloom* (o brilho "vazando" ao redor de luzes) usam um pipeline extra (`EffectComposer`). É o que dá aquele acabamento de "jogo AAA" nos elementos neon — vale como etapa final, só depois de tudo funcionar sem ele.



Mesmo layout de cena — o polimento vem de sombra, brilho (emissivo) e neblina, não de geometria nova.

Exemplo ilustrativo

```
// sombras
renderer.shadowMap.enabled = true;
sun.castShadow = true;
floor.receiveShadow = true;
robot.traverse(obj => obj.castShadow = true);

// neblina - cor deve combinar com o fundo da cena
scene.fog = new THREE.Fog(0x0b1220, 15, 45);

// material emissivo (efeito neon)
const neonMat = new THREE.MeshStandardMaterial({
  color: 0x111111, emissive: 0x7fd8b0, emissiveIntensity: 1.5
});

// responsividade
```

```
addEventListener('resize', () => {  
  camera.aspect = innerWidth / innerHeight;  
  camera.updateProjectionMatrix();  
  renderer.setSize(innerWidth, innerHeight);  
});
```

APLICAÇÃO NO SEU PROJETO

Priorize nesta ordem, se o tempo for curto perto da entrega: **1)** responsividade (funcionar bem na tela do projetor da escola é mais importante que qualquer efeito), **2)** sombras (maior ganho visual pelo menor esforço), **3)** fog + emissive para reproduzir a atmosfera do print de referência, **4)** bloom/pós-processamento só se sobrar tempo. Esta é também a etapa certa para revisar o conteúdo dos popups com atenção — texto claro e sem erros pesa tanto quanto o visual na avaliação de um projeto escolar.

Aprofunde

threejs.org/docs → "Fog", "MeshStandardMaterial.emissive", "shadowMap". Exemplo oficial (bloom):
threejs.org/examples/#webgl_postprocessing_unreal_bloom.

Checklist rápido antes de apresentar

- ☐ O robô se move de forma suave em qualquer computador (delta time aplicado)
- ☐ Câmera nunca perde o robô de vista, mesmo nos cantos do mapa
- ☐ Todas as estações do currículo têm popup funcionando
- ☐ Textos dos popups revisados (sem erro de português/técnico)
- ☐ Tela testada em pelo menos duas resoluções diferentes (notebook + projetor)
- ☐ Nenhum objeto "flutuando" sem sombra/contato visual com o chão

Onde buscar ajuda quando travar

Documentação e comunidade

- threejs.org/docs — referência de toda classe/método citado neste documento
- threejs.org/examples — exemplos oficiais com código-fonte navegável
- discourse.threejs.org — fórum oficial, ótimo para buscar erros específicos
- **Three.js Journey (Bruno Simon)** — canal/curso do criador de um portfólio 3D muito próximo do seu conceito (robô/carro dirigindo por um mapa)